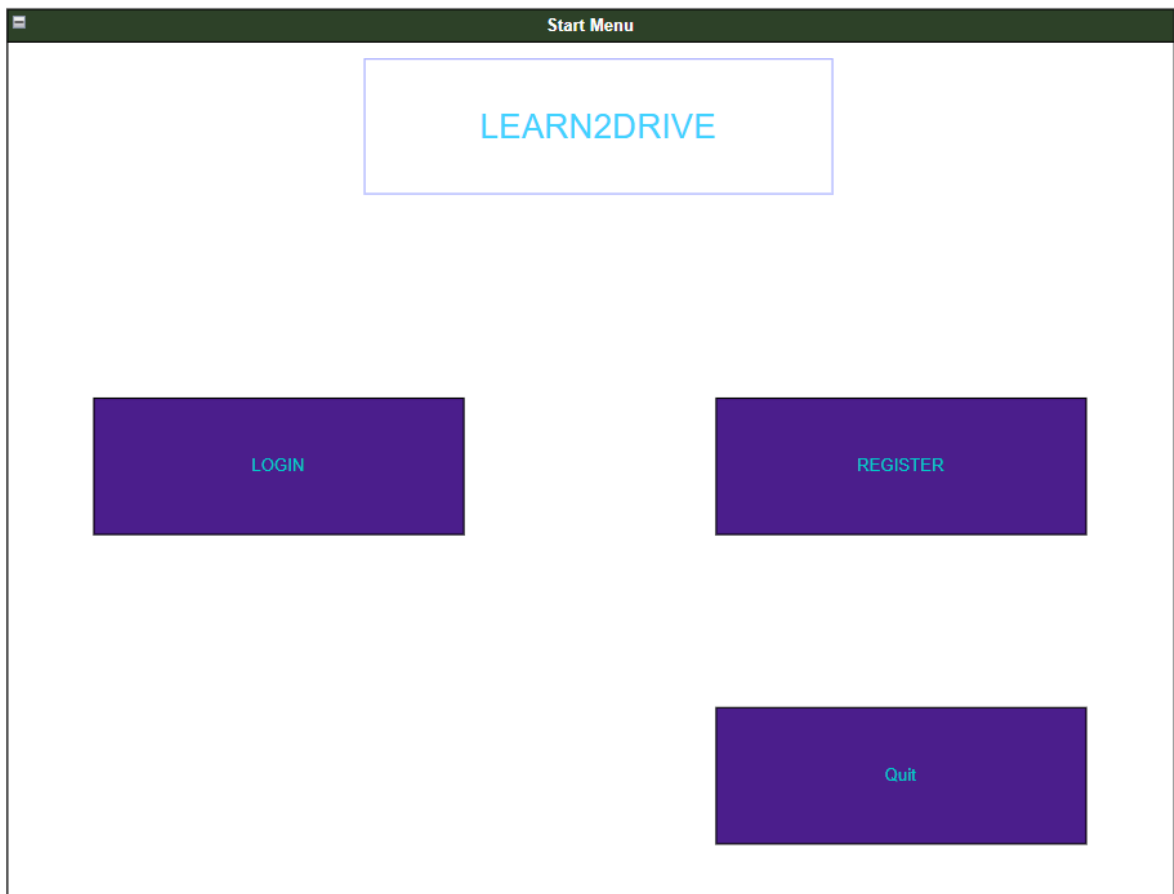
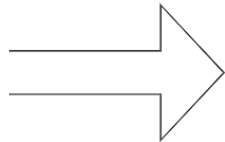


Interface diagram





Register

UserName

Conditional statement--> *USERNAME TAKEN*

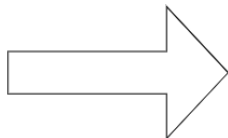
Password

MUST HAVE:

8 characters
1 number
1 Special Character (E\$%&!*)

Conditional statement--> *Password doesnt Meet Requirements*

Sign up



LOGIN

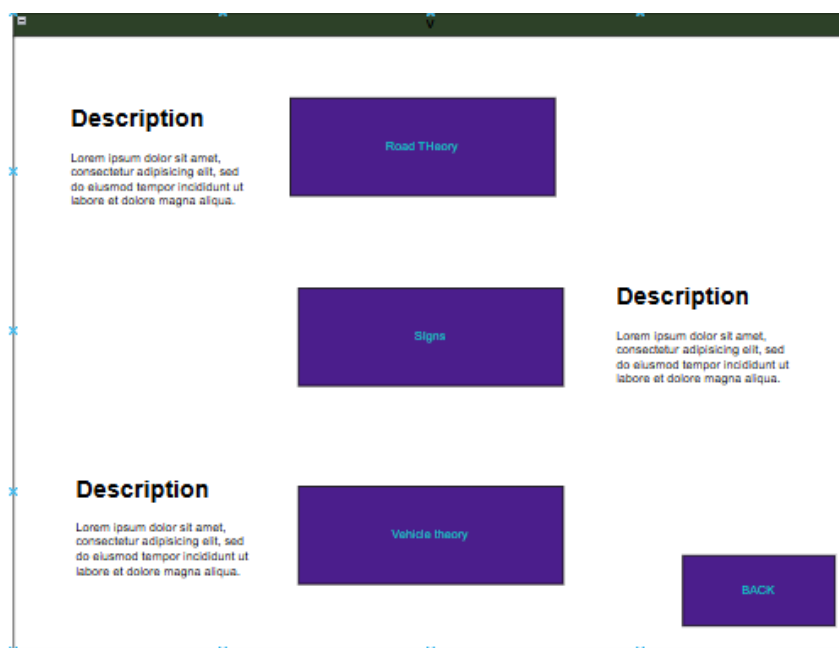
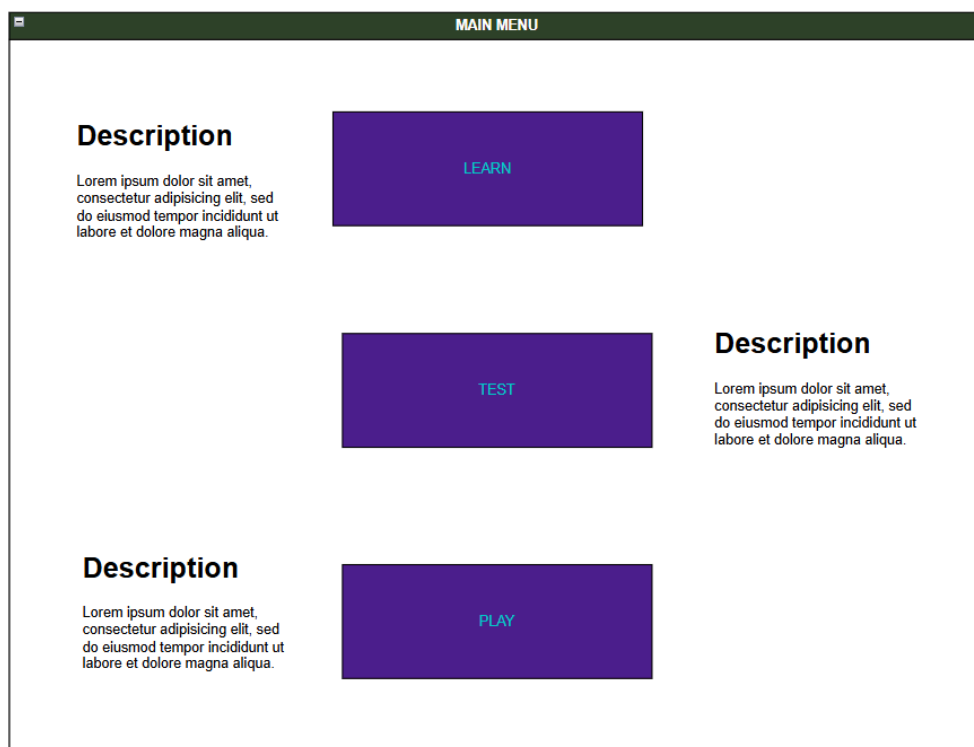
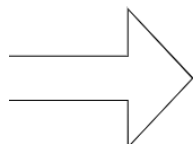
UserName

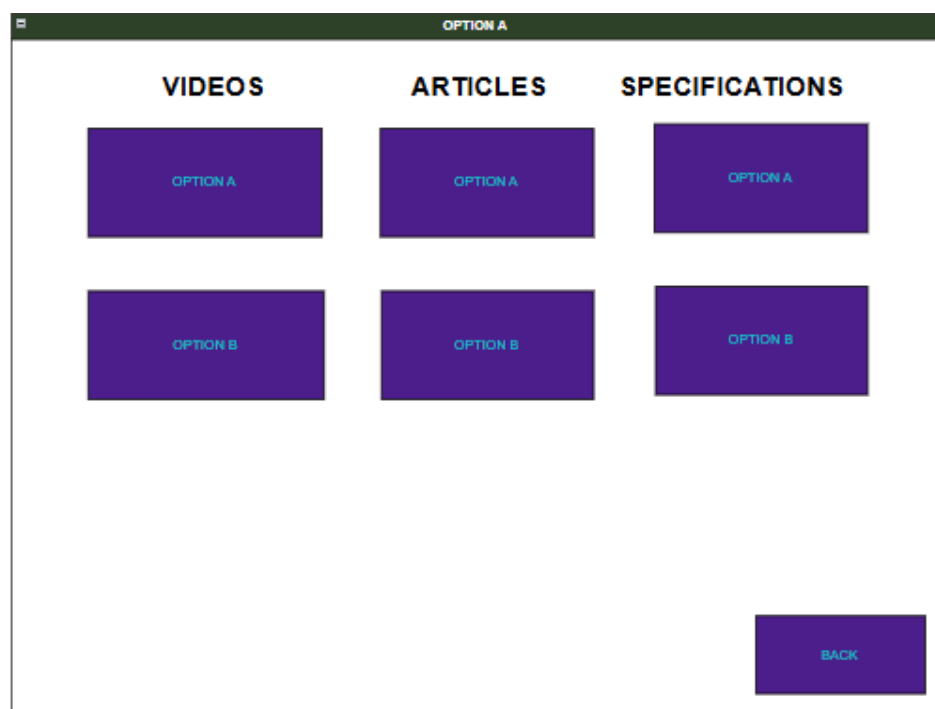
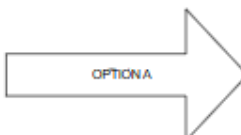
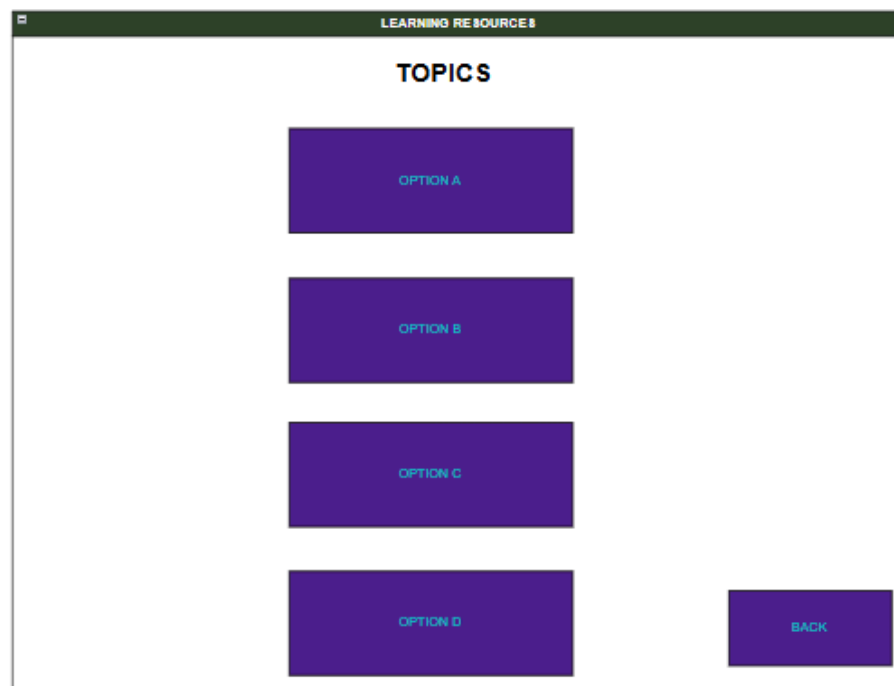
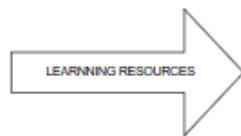
Conditional statement--> *Incorrect Username*

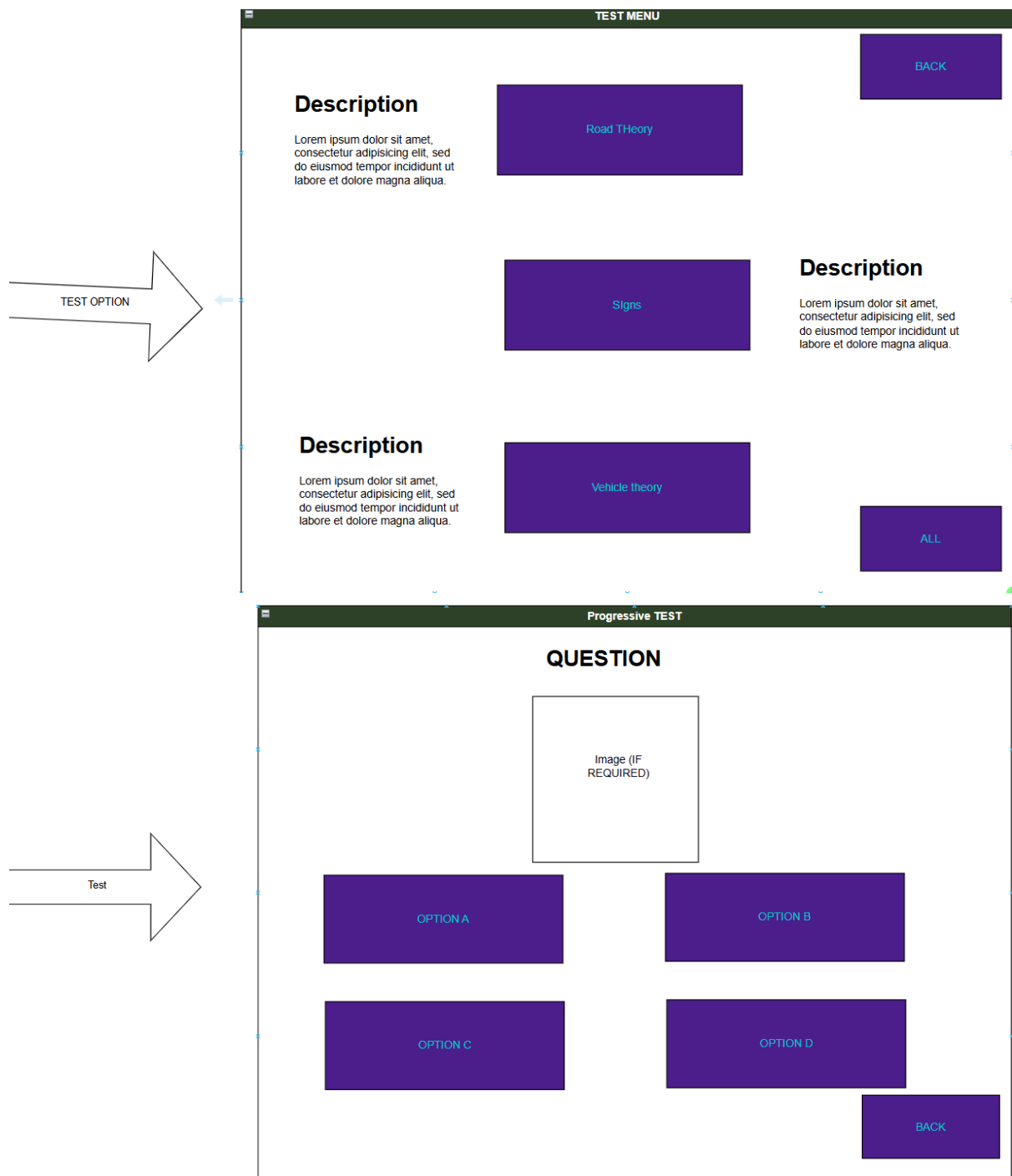
Password

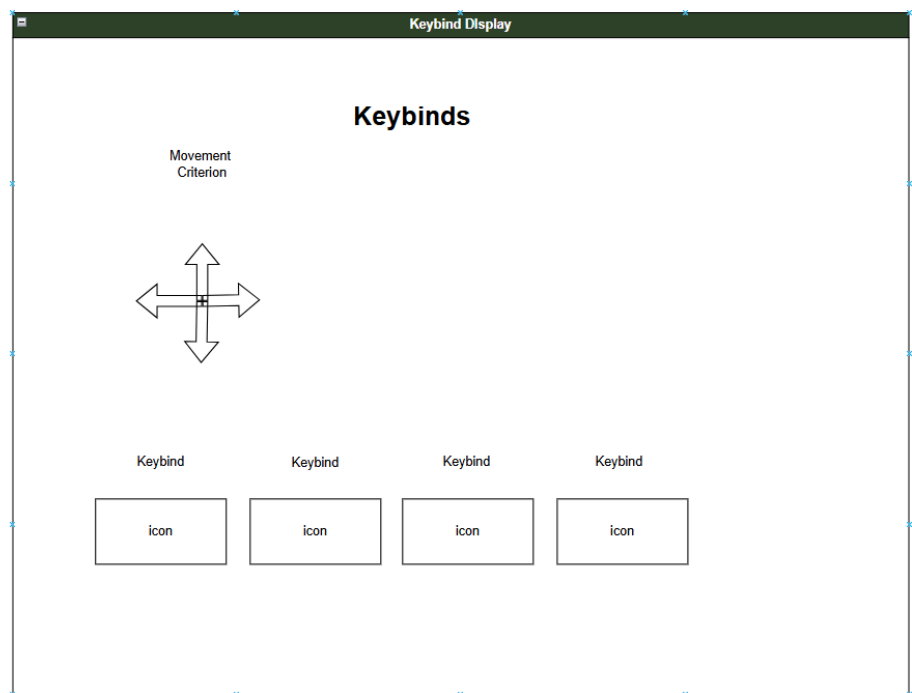
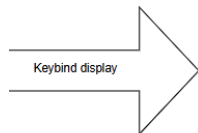
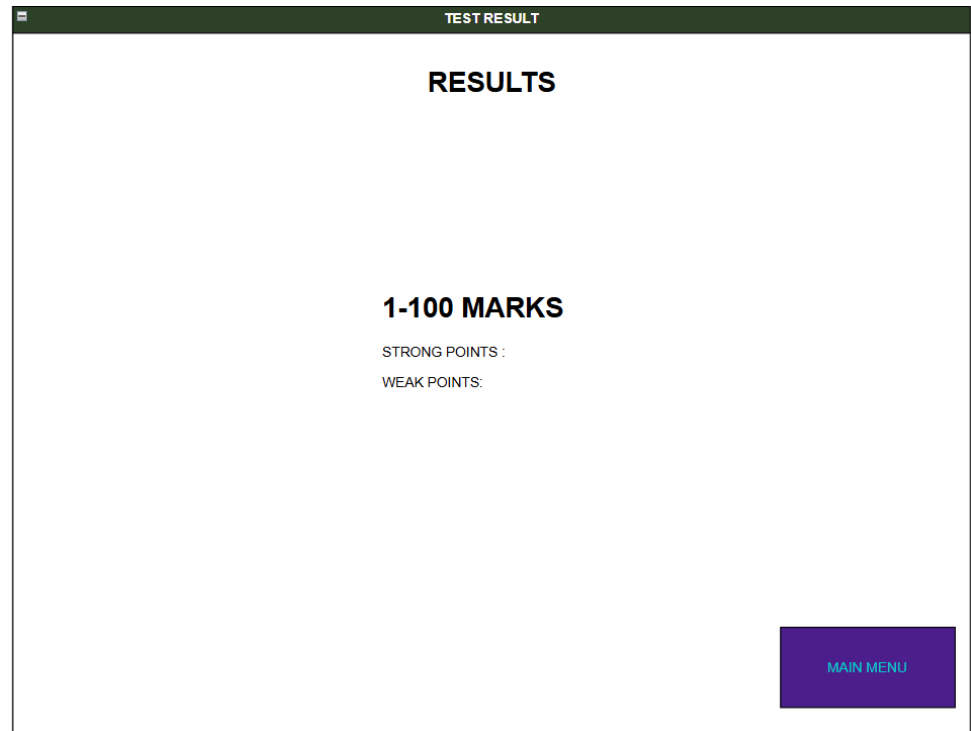
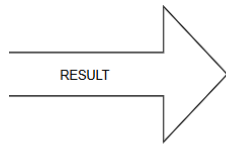
Conditional statement--> *Password doesnt match*

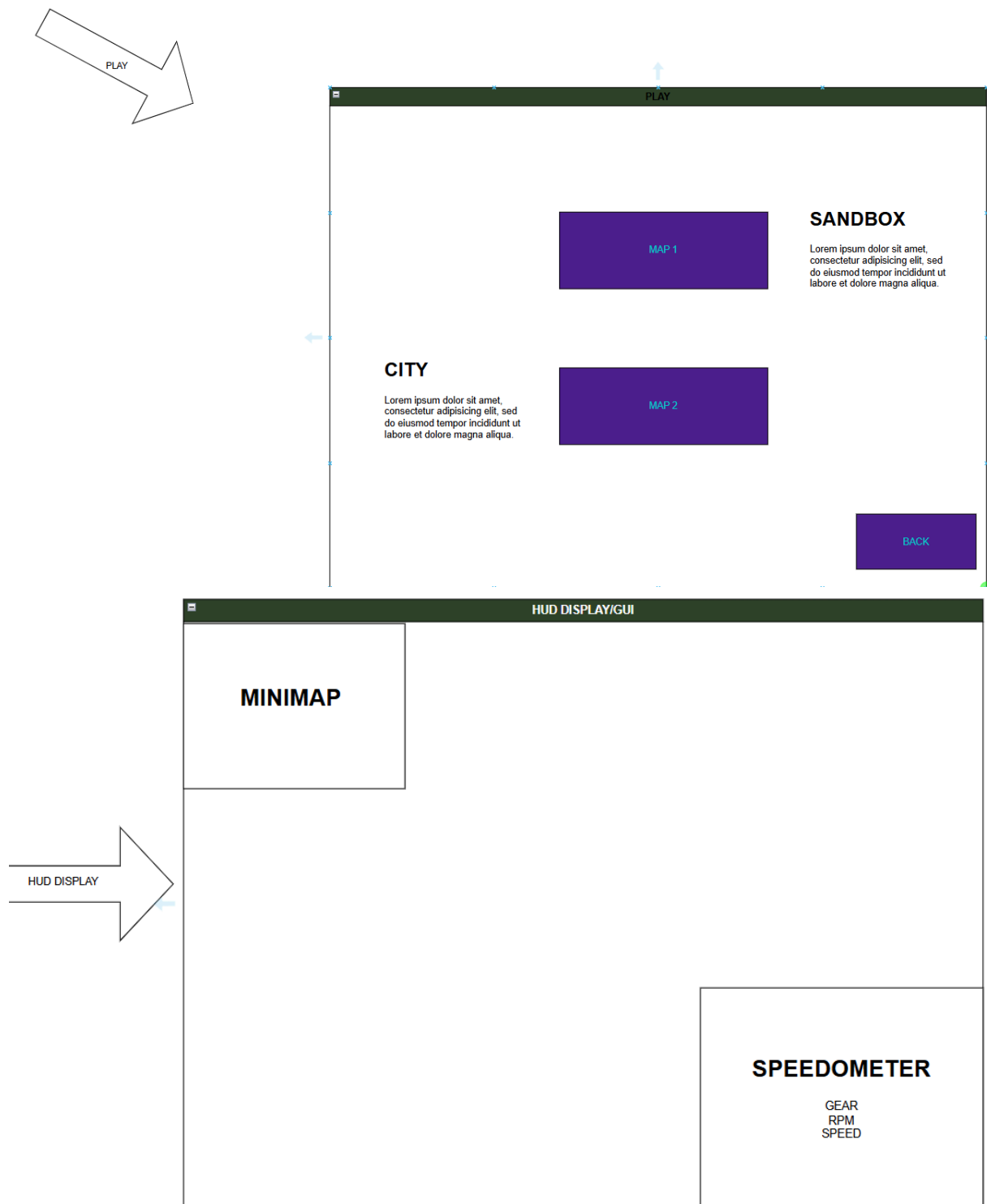
Sign up







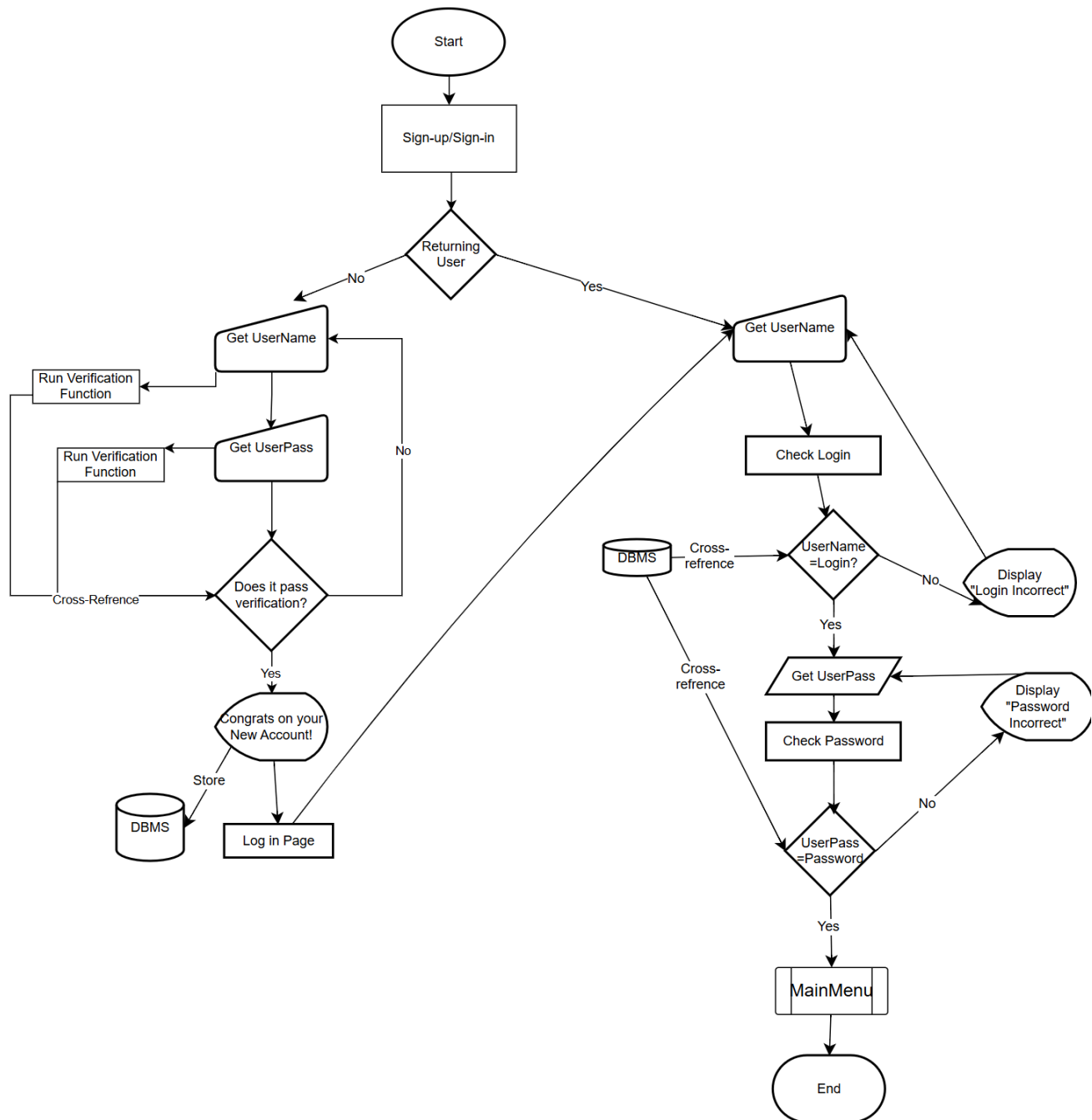




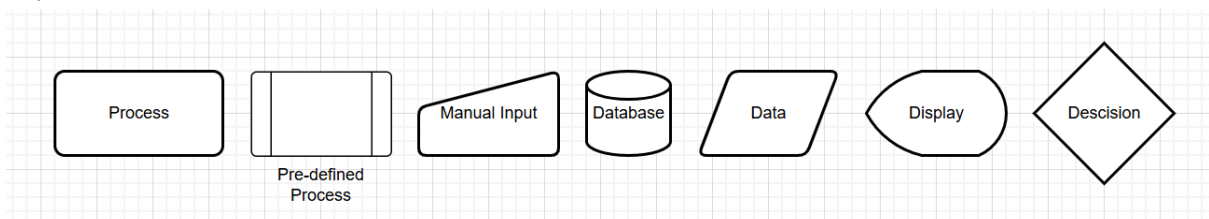
Database dictionary:

Field Name	Data type	Field Length	Constraint	Description
Client_name	string	25	not null	client name
Password	string	25	not null	log in password for client
Time_of_Login	int	4	not null	time of login/activity
_id	string	40	primary key	Client_id,automatically generated

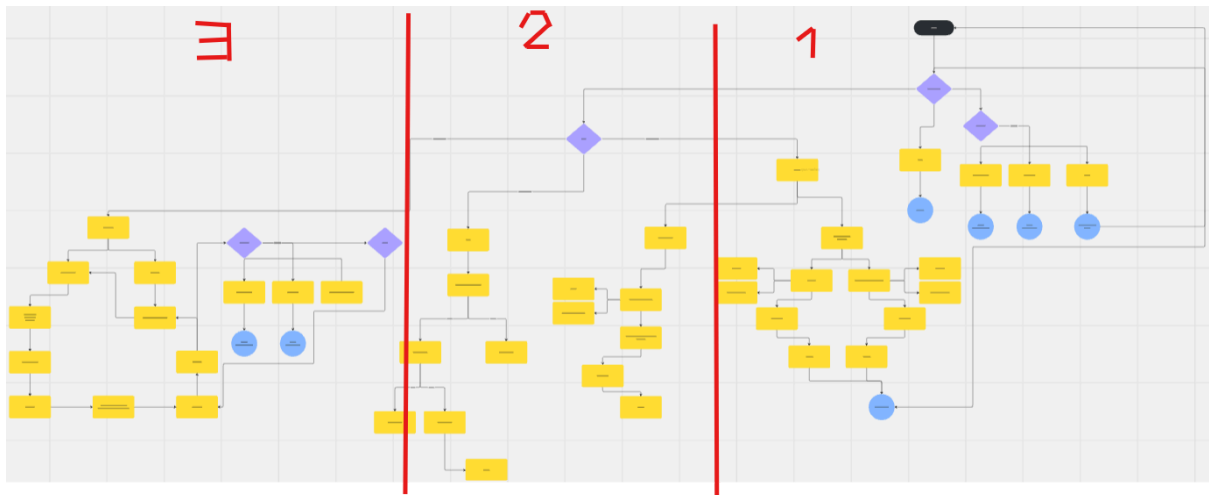
UML

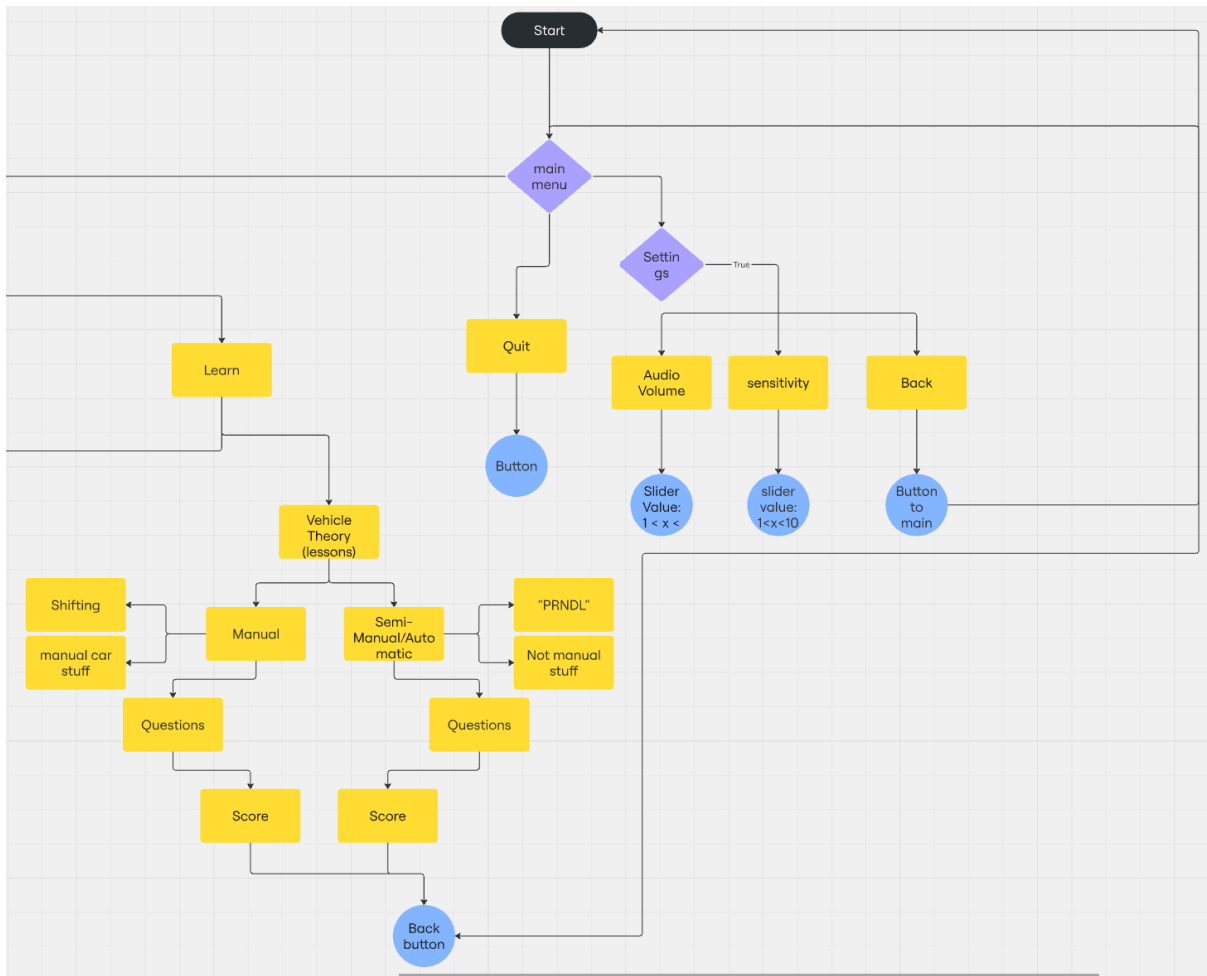


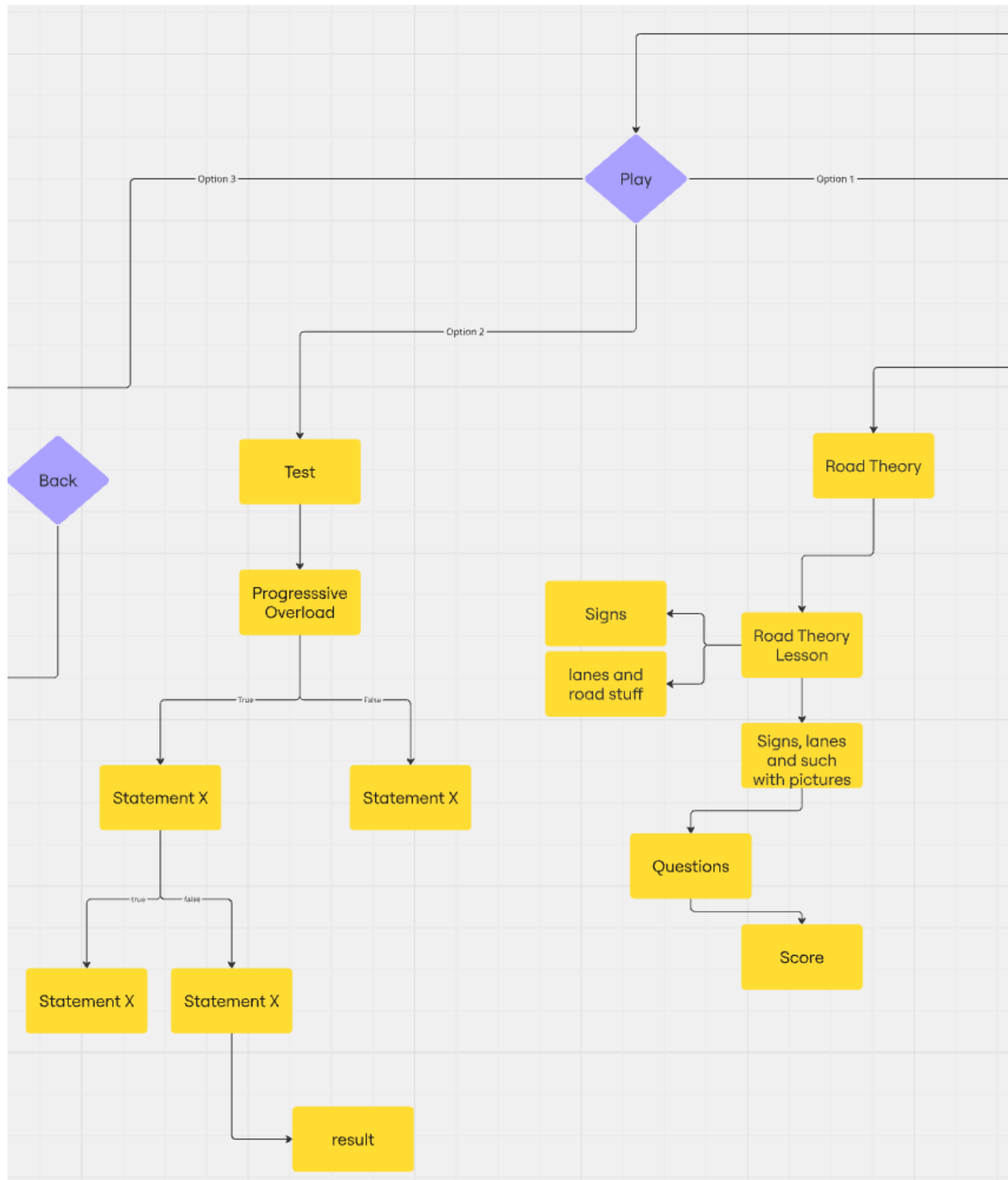
Key:

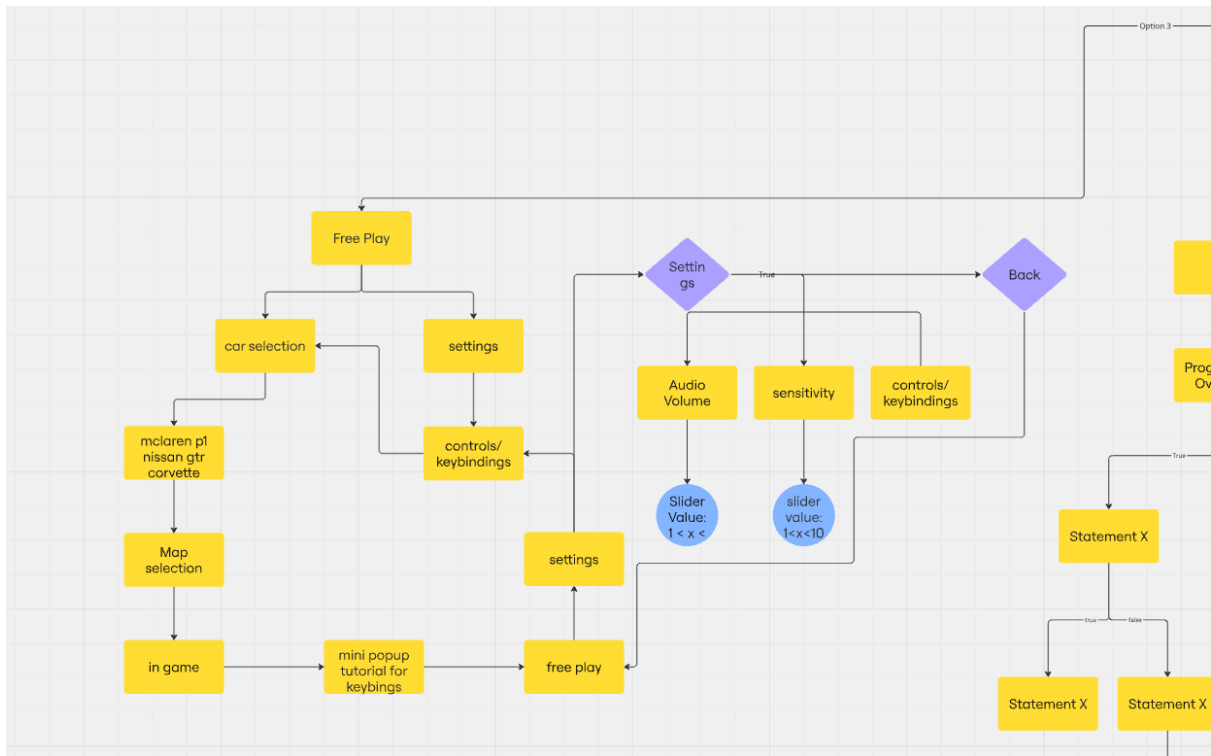


Program structure chart

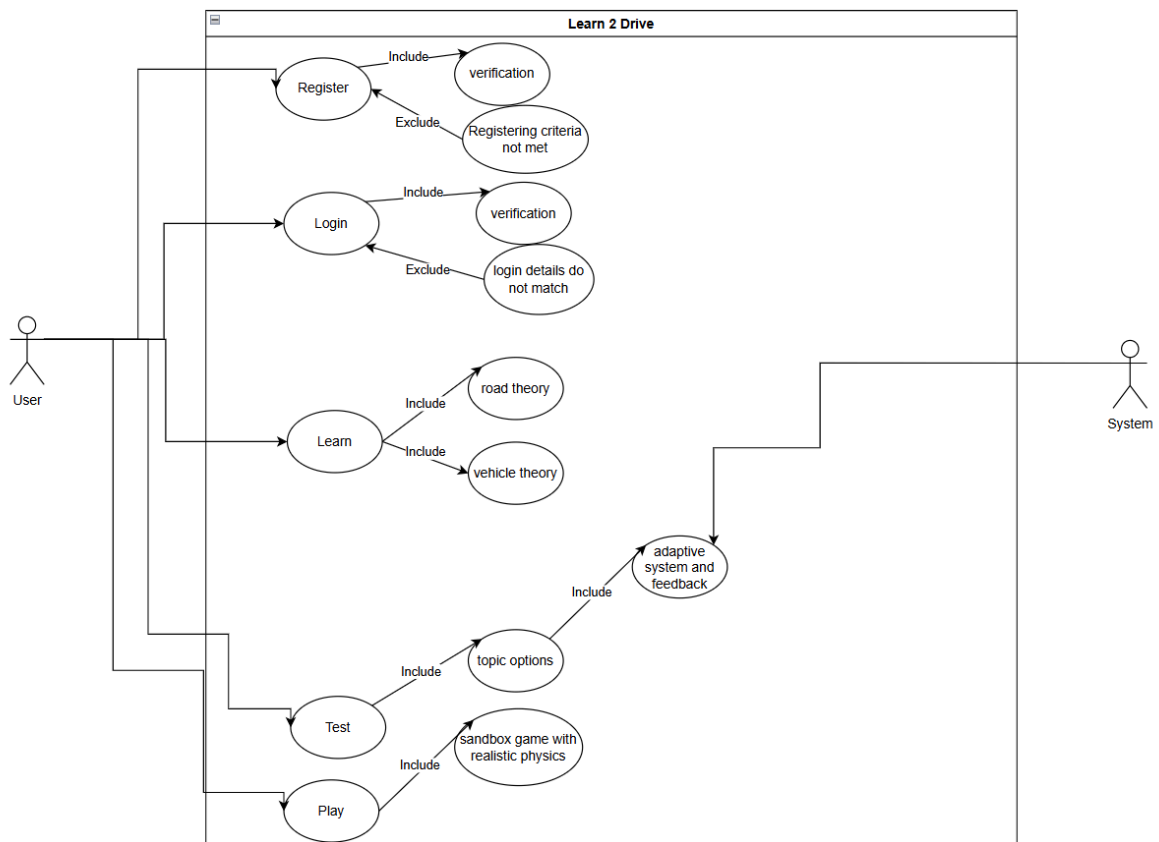




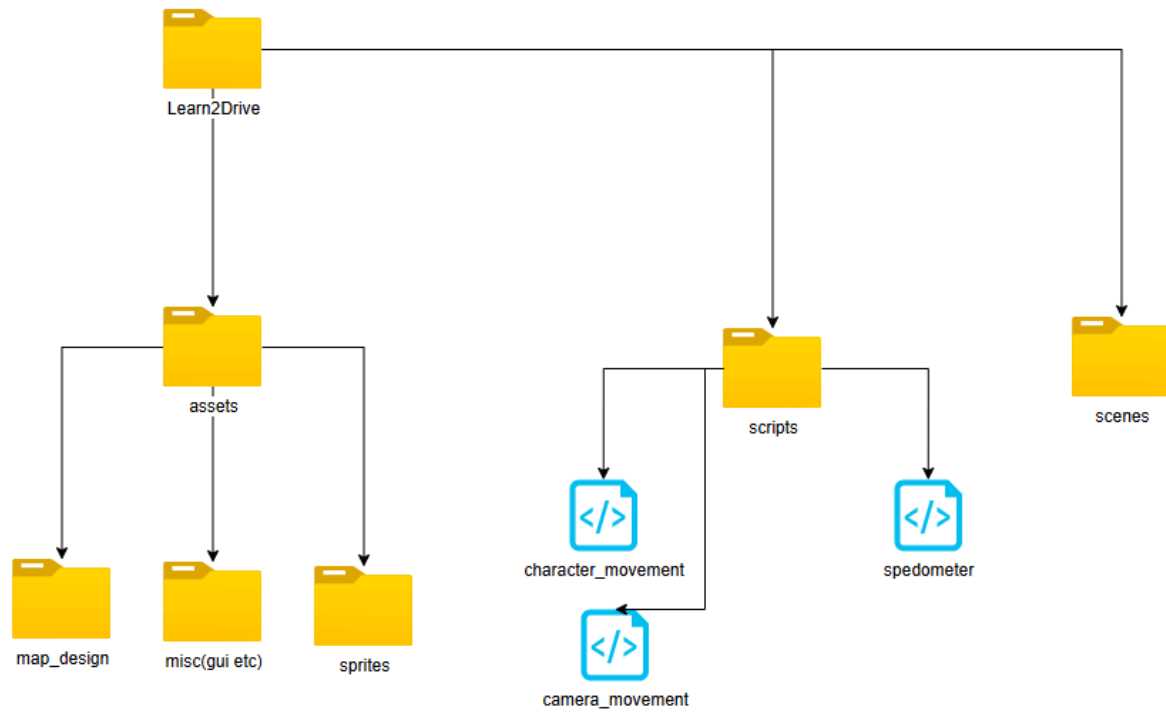




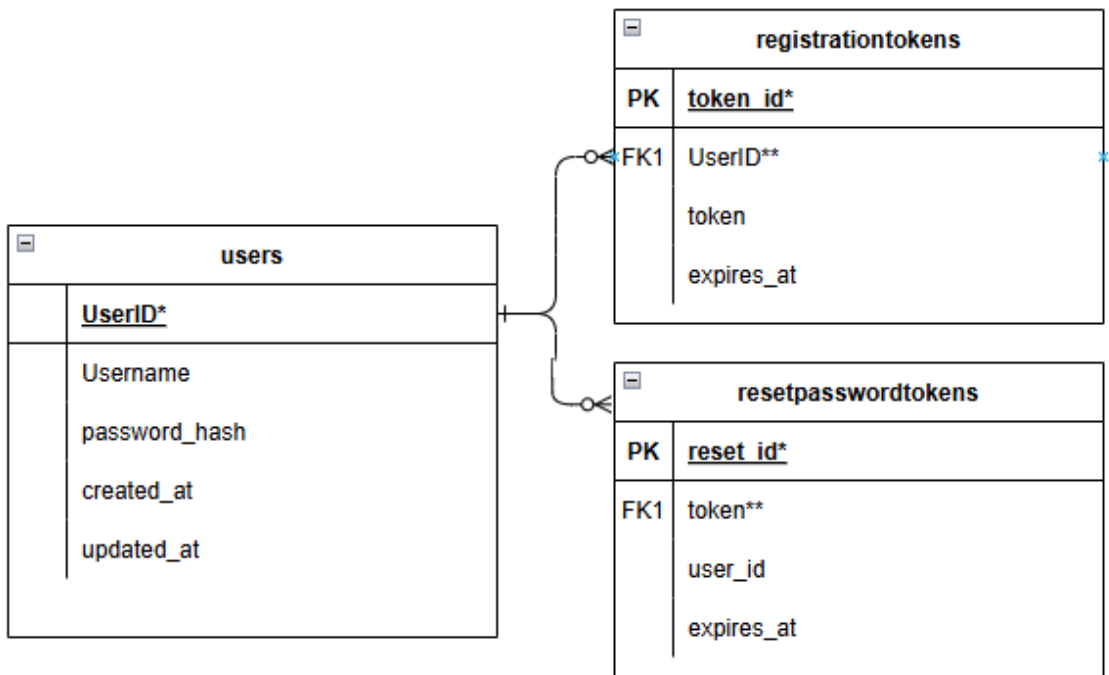
Use-case diagram



Filing structure



Relationship diagram



Test Plan

DESIGN SPECIFICATIONS	Actions to test	Testing methods
	I. The application should include a way for the user to test out their knowledge practically. E.g. simulation	
	Drive the vehicle and test steering response	Functional testing of steering input
	Accererate and brake the vehicle	Functional testing of acceleration and braking mechanics
	Drive into obstacles to trigger collision	Collusion detection testing
	II. The program should offer environment(s) that simulate real world dynamics	
	Load and enter 2 environments	Environment testing and validation testing
	Drive around in both environments	Usability testing of environment navigation
	III. The program should include realistic physics and elements (i.e lights skid marks etc.)	
	Drive at high speed and brake suddenly	Physics stress test
	Turn on headlights and check visibility	Feature functionality testign
	Perform sharp turns to observe skid makrs	Physics and visual feedback testing and providing feedback (skid marks
	IV. The program should include a area for users to test their knowledge	
	Enter testing section	UI testing for module acess
	Answer questions and submit responses	Functional testing of question input and submission
	V. The users should be given the option of topics they want to cover in said tests	

	Select a topic to test	Input validation testing for topic selection
	Start a test with that topic	Functional testing of filtered question sets
	VI. The program should include progressive difficulty that adjusts difficulty	
	Answer questions correctly in sequence	Algorithm testing of difficulty increase
	Answer question incorrectly	Algorithm testing of difficulty decrease
	VII. Users should be able to have a choice in customizing their experience and learning. E.g different topic focuses	
	View multiple learning topics	Functionality testing
	Select a learning topic	UI testing for topic selection
	VIII. The application offers separated modules for road signs, driving laws and rules	
	View learning modules	UI testing with topic selection
	Select a learning module	UI and Functionality testing.
	IX. The application includes a working sign in/up page	
	Attempt to sing in with invalid credentials	Authentication input validation testing
	Attempt to sign in with valid credentials	Function testing of login success and redirection